NightShift — Product Requirements Document (PRD)

NightShift — Product Requirements Document (PRD)

Google & Kaggle 5-Day AI Agents Intensive (Vibe Coding) — Capstone Track: Agents for Business

> *Companion document:* ***NightShift — Technical Design Document (TDD)****, covering architecture, code structure, and implementation detail. This PRD covers the product "what and why"; the TDD covers the "how." Cross-references to "the TDD" throughout this document point to that companion file.*
>
> *Revision: v10 (split) — this is the PRD half of a combined v10 PRD/TDD suite, split into two standalone files at the user's request. Content is unchanged from v10; only the file boundary and a few cross-references changed.*

SECTION 0: RUBRIC TRACEABILITY MAP (verified against Kaggle's official Overview page)

This map exists so that, when you write the Kaggle Writeup, you can point a judge at a specific section of either document for every point category — nothing in the rubric should require explanation you haven't already written down.

Rubric item	Points	Where it's satisfied
Core Concept & Value	10	This PRD, §1.1 (value proposition), §1.3 (success metrics tie the agent's existence to a measurable business outcome)
YouTube Video (≤5 min)	10	§3.2 below — Video script outline, 5 required beats
Writeup (≤2,500 words)	10	§3.1 below — Writeup outline; content drawn from this PRD §1 + the TDD §2
Technical Implementation	50	**TDD** §2.1–§2.5 in full; graph topology specifically in TDD §2.2; optional deployment evidence also counts here, not as a separate category
Documentation (README.md)	20	§3.4 below — README structure: problem, solution, architecture, setup, diagrams
Key Concepts checklist — need 3 of 6, this build hits 5 of 6	—	Agent/Multi-agent (ADK) → Code: TDD §2.2; MCP Server → Code: TDD §2.2; Security features → Code: TDD §2.4; Agent skills/Agents CLI → Code: TDD §2.3; Deployability → Video (optional, mentioned): §3.2. *Antigravity → Video is the one item not naturally hit, since this build uses Cursor — see §3.2 for how to address it on camera anyway.*

Required submission assets (all four mandatory, not just the Writeup): Kaggle Writeup · Media Gallery (cover image required) · Attached public YouTube video · Public Project Link (working demo, or a public GitHub repo with setup instructions if a live demo isn't feasible).

Deadline: July 6, 2026, 11:59 PM PT. Draft/unsubmitted Writeups at the deadline are not considered.

🚨 **Do not include any API keys or passwords in your submitted code** — this is an explicit, named disqualifying risk on the official page, not just a general best practice.

SECTION 0.5: COMPETITIVE POSITIONING & DIFFERENTIATION

Honest framing, stated up front: an overnight ingest → classify → draft → human-approves agent is structurally the same shape as the course's own Day 4 worked example (the expense-approval agent). In a pool that may run into the thousands, especially in "Agents for Business," many submissions will be some variant of this pattern across different domains (expenses, support tickets, HR requests). Architectural correctness alone — using ADK, MCP, memory the way the course teaches — makes a submission competent, not distinctive. Distinctiveness has to be added deliberately. This section exists so that decision is made once, explicitly, rather than left implicit and diluted across five competing claims.

0.5.1 The headline hook (pick one, lead with it everywhere)

Of NightShift's real strengths — domain specificity, HITL safety architecture, multi-agent design, memory — **the HITL safety architecture is the strongest standalone hook**, for one concrete reason: it's the easiest one to make *visceral* in 5 minutes of video, and it's the one most other "approval agent" submissions will treat as a minor footnote rather than the headline. Everyone building an approval-queue agent will *say* "human stays in control." Few will *show* that the guarantee is enforced at the database constraint level, immune even to a bug or a compromised agent.

The headline, stated as it should appear in the Writeup title/subtitle and the first 10 seconds of the video:

> *"NightShift drafts. It never sends. Not because the prompt says so — because the database won't let it."*

This reframes a safety feature (often a checkbox item judges skim past) into the actual selling point. It also directly answers the "why agents, specifically" beat (§3.2) better than a generic efficiency pitch would: the answer becomes "because the overnight inbox needs judgment, and judgment needs a human backstop that can't be bypassed by an agent having a bad night."

0.5.2 The domain-specificity layer (supporting evidence, not a second headline)

Don't compete on "businesses need automation" — that's the crowded, generic claim every submission will make. Compete on demonstrated domain judgment: NightShift's RED/YELLOW/GREEN matrix isn't generic urgency scoring, it's tied to real property-management failure modes (active city fines, structural damage, compounding daily penalties for an unaddressed code violation). Naming a concrete real-world consequence — even one illustrative dollar figure or scenario — in the Writeup's problem statement does more work than a paragraph of generic value language. Use this as supporting evidence for the headline hook, not as a competing pitch; one strong claim beats two medium ones.

0.5.3 The single best demo moment (build this specifically, don't leave it to chance)

A checklist-complete demo (show ingestion, show classification, show a draft, show approval) proves competence but isn't memorable. One ambiguous, genuinely hard classification case is worth more screen time than three easy ones. Concretely: script and seed a test item that's deliberately borderline — for example, a tenant email reading *"the ceiling above my bathroom has a small water stain, has had it for a week, nothing dripping yet"* — which could plausibly be filed GREEN (no active leak, no emergency) or RED (early sign of structural water damage, the kind of thing that becomes a city violation if ignored). Show `TriageAgent`'s actual reasoning trace landing on RED with its stated rationale, then show the manager's morning view surfacing *why* it was escalated. This single moment demonstrates judgment, not just classification — and is the kind of specific, replayable beat judges remember after watching dozens of submissions in one sitting.

0.5.4 Track decision — make it final, not hedged

Two tracks both fit NightShift legitimately:

- **Agents for Business** — the default, obvious fit (workflow automation, cost/revenue on the line via avoided fines and vendor errors).
- **Concierge Agents** — track description explicitly emphasizes "safe and secure agents" that "free time for things that really matter" and keeping "personal information safe and secure." NightShift's per-tenant PII scoping and the

headline HITL guarantee map onto this framing at least as well, and the applicant pool for this track is likely smaller than the flagship “Agents for Business” track.

Decision for this document: submit under Agents for Business, on the reasoning that property management is unambiguously a business operations problem and a judge reading a Concierge Agents submission about commercial property management may feel it’s a mismatch regardless of the safety angle — track mismatch risk outweighs the smaller-pool benefit here. State this choice once in the Writeup and don’t hedge between tracks in the text itself (§3.1).

SECTION 1: PRODUCT REQUIREMENTS DOCUMENT (PRD)

1.1 Core Value Proposition & Objective

NightShift drafts. It never sends. Not because the prompt says so — because the database won’t let it. (See §0.5.1 for why this is the headline, not just a feature.)

NightShift is an overnight AI assistant for property managers. While the manager sleeps, it ingests every overnight communication — city notices, tenant reports, HOA messages, vendor invoices, and inspection alerts — from multiple disconnected sources, and produces a single, ranked **Morning Brief** by the time the manager wakes up.

Primary objective: automate multi-source ingestion and triage end-to-end, while guaranteeing **0% automated outbound action without explicit human sign-off.** NightShift drafts; it never sends.

Why this matters, concretely: a missed or mis-prioritized overnight item in property management isn’t just lost time — it can be a compounding daily city fine, an unaddressed structural issue that worsens overnight, or a tenant escalation that becomes a legal matter by morning. Property managers currently lose the first 30–90 minutes of each morning manually re-reading scattered inboxes and portals just to figure out what’s urgent, with no guarantee the most dangerous item was even read first. NightShift compresses that into a five-minute review — and the item most likely to cost real money or create real liability is always at the top, not buried in inbox order.

1.2 Non-Goals (Out of Scope)

Explicitly excluded from v1, to keep the engineering surface bounded and to make the safety guarantees auditable:

- NightShift does **not** place phone calls or handle voice channels.
- NightShift does **not** negotiate vendor pricing or terms — it flags billing anomalies, it doesn’t resolve them.
- NightShift does **not** auto-send under any circumstance, including high-confidence GREEN classifications. Every draft, regardless of urgency tier, sits in staging until a human approves it.
- NightShift does **not** make legal determinations about city notices (e.g., whether a fine is valid) — it surfaces and routes, a human or counsel decides.
- NightShift does **not** manage multi-night-shift handoffs across time zones in v1 (single property-manager-per-portfolio assumption).

1.3 Success Metrics

Metric	Target	Why it matters
Triage accuracy (RED/YELLOW/GREEN vs. human relabeling)	≥ 90% agreement	Core trust signal — if classification is wrong often, the brief isn’t actionable
False-RED rate (over-escalation)	< 15%	Too many false alarms erodes manager trust faster than under-escalation
False-GREEN rate (under-escalation)	< 2%	This is the dangerous failure mode — a missed structural/legal issue
Time-to-Morning-Brief	< 10 min for a 50-item inbox	Brief must be ready before the manager’s first coffee, not during it
Per-item processing latency	< 5 sec/item (Flash classification call)	Operational health metric — distinguishes a slow model call from a slow tool call, see TDD §2.5 trace attributes

Memory lookup success rate	≥ 98% (tenant → property resolved without fallback)	A low rate here means the memory schema is incomplete, not that the LLM is failing — useful for distinguishing failure causes
Draft acceptance rate (approved with no edits)	≥ 60%	Measures whether drafts are actually useful or just busywork to edit
Human-approval turnaround time	Tracked, no hard target in v1	How long items sit in `staged` before a manager acts — useful operational signal even though NightShift can't and shouldn't try to influence it
Token cost per overnight run	Tracked, budgeted	Keeps the system commercially viable at scale
Auditability	100% of classifications traceable via OpenTelemetry	Directly demonstrates the Day 5 observability standard — every RED/YELLOW/GREEN call should be explainable after the fact, not just in aggregate

1.4 User Personas & Workflows

Persona: Property Manager ("Morning Reviewer") — manages 5–40 units/properties, checks NightShift's brief as the first task of the day, has final authority over every outbound message.

Workflow A — Overnight Ingestion (no human present):

1. NightShift session initializes at a scheduled time (e.g., 11 PM local).
2. Pulls new items from each connected source since last run.
3. Classifies, matches to property, drafts response where applicable.
4. Writes all output to a staging store. No external message is ever sent during this phase.

Workflow B — Property Manager Morning Approval (human present):

1. Manager opens the Morning Brief — items grouped by urgency, RED first.
2. For each item: read summary → read draft response (if any) → **Approve / Edit & Approve / Reject / Snooze.**
3. Only on explicit Approve does a message move from staging to "ready to send" — and per Non-Goals, actual sending in v1 is itself a manual action the manager confirms (auto-send is not implemented even post-approval, until the production hardening phase explicitly enables it under its own audited flag).

1.5 Functional Scope & Rules

Urgency classification matrix:

Tier	Definition	Example triggers
RED	Structural damage, active city fines, life-safety issues	"no heat," "code violation notice," "water leak ceiling"
YELLOW	Vendor billing anomalies, upcoming inspections, time-sensitive but non-emergency	invoice total mismatch, inspection date within 7 days
GREEN	Standard tenant maintenance updates, routine HOA notices	"lightbulb out," general newsletter from HOA

The hard case (build this as a named test fixture, not an afterthought): the matrix above is easy on clean examples and that's exactly the risk — a judge skimming code sees three tiers and three obvious example phrases and concludes "fine, but unremarkable." The fixture that actually demonstrates judgment is a deliberately ambiguous item: *"the ceiling above my bathroom has a small water stain, has had it for a week, nothing dripping yet."* No active leak (arguably GREEN), but an early indicator of structural water damage that becomes a city violation if ignored (arguably RED). This fixture is referenced again in §0.5.3 (demo strategy) and §3.2 (video script) — build and seed it early so the same scenario can be reused across the README test suite, the demo, and the video without extra work.

Human-in-the-Loop (HITL) constraint: every drafted communication is created in a `staged` state. State can only transition to `approved` via an explicit manager action (web UI click or CLI confirm) carrying their identity and a timestamp. No code path exists that transitions `staged → sent` without passing through `approved` first — this is enforced at the data layer (see the TDD, §2.5, Human Triage State Machine), not just in application logic, so a bug in the agent's reasoning cannot bypass it.

SECTION 3: SUBMISSION DELIVERABLES (40 rubric points combined — Writeup, Video, Documentation)

Four assets are mandatory for a valid submission, confirmed from Kaggle's official page: **Kaggle Writeup, Media Gallery (with a required cover image), an attached public YouTube video, and a Public Project Link.** Missing any one of these makes the submission invalid regardless of code quality — these aren't optional polish.

3.1 The Kaggle Writeup (10 points, ≤2,500 words — hard limit, overage may be penalized)

This is your project report, not a duplicate of the PRD/TDD — keep it tight against the word cap. **Target ~1,800–2,100 words**, not the full 2,500 — leaving headroom avoids a last-minute cut-down under deadline pressure and reads tighter to a judge skimming many submissions in one sitting. Recommended structure, each piece pulled directly from these two documents so there's no last-minute re-explaining:

1. **Title and subtitle** — build directly from the headline hook (§0.5.1): title "NightShift," subtitle **"NightShift — drafts overnight, never sends without you."**
2. **Problem statement** (2–3 sentences) — from §1.1, using the concrete failure-mode framing (compounding fines, worsening structural issues), not generic efficiency language.
3. **System architecture graph** — the diagram from TDD §2.2, with a one-line caption per agent.
4. **What it does, end to end** — Workflow A + B from §1.4, condensed to a short narrative.
5. **The hard case** — briefly walk through the ambiguous water-stain fixture (§1.5 / §0.5.3) in text, even though it's also shown in the video; a judge skimming the Writeup without watching the video yet should still see the judgment-call evidence.
6. **Key Concepts demonstrated** — copy the checklist table from §0, filled in with a one-line "how" for each of the 5 you hit.
7. **Engineering journey / decisions** — pull 2–3 rows from the Architecture Decision Log (TDD §2.7) verbatim; judges score "your project's journey," so a real decision and its tradeoff (e.g. the multi-agent choice) is worth more than a clean narrative with no visible reasoning.
8. **Project link** — your Public Project Link (§3.3), restated here for convenience even though it's also submitted as its own field.

Track: Agents for Business, per the final decision in §0.5.4 — state it once, don't hedge between tracks in the text.

3.2 The YouTube Video (≤5 minutes, 10 points, must be published to YouTube and attached to the Media Gallery)

Structural note: the five official beats (problem, why agents, architecture, demo, the build) are all required and all covered below — but they're not treated as five equal, independent segments. They're built around one continuous thread: the headline hook from §0.5.1, proven through the one hard demo case from §0.5.3. A checklist-complete video that covers every beat competently is necessary but not sufficient to stand out in a large pool — the goal here is for a judge to remember a specific moment after watching many submissions, not just confirm all five boxes were ticked.

Segment	Target length	Content
Hook + problem statement	~45 sec	Open with the headline line itself, spoken: *"NightShift drafts. It never sends. Not because the prompt says so — because the database won't let it."* Then the overnight-inbox pain point with the concrete failure

		mode (a missed early water-damage sign becoming a city violation), not generic "businesses need automation" framing. **Optional cheap addition:** a 5-second before/after split-screen here — chaotic mock inbox on the left, the clean ranked Morning Brief on the right — is inexpensive to produce and gives the video an immediate visual hook before any narration even starts
Why agents	~30 sec	Direct answer: this task requires reading unstructured, ambiguous text and making a judgment call a fixed rules engine can't — which is exactly what gets proven in the demo segment below, so this beat should explicitly forward-reference it ("watch what happens with a genuinely ambiguous case in a minute") rather than standing alone
Architecture	~50 sec	Walk through the TDD §2.2 graph on screen — name the three agents, the MCP/memory connections — kept tight since the demo is where the real screen time goes
The hard-case demo	~2 min	This is the segment that should get the most time and the most rehearsal. Run the ambiguous water-stain item (§1.5) through the live system. Show `TriageAgent`'s actual reasoning trace landing on RED — script the exact on-screen text rather than leaving it to whatever the model happens to output live, e.g.: *"Reasoning: 'small water stain… for a week' matches early structural water-damage pattern (RED). Even without active drip, risk of code-violation escalation within days is high."* Then cut to the manager's Morning Brief showing that same rationale surfaced, and the Approve/Reject/Snooze action — closing the loop back to the headline hook by explicitly showing the draft sitting in `staged`, untouchable, until that click happens
The Build	~25–35 sec	Name ADK 2.0, Gemini API, MCP, and the tools used. Explicitly mention Antigravity here (e.g. "the course's reference IDE is Antigravity; I built in Cursor as an equivalent agentic IDE") since it's graded as its own Key Concept item via video and isn't otherwise touched

Rehearse the hard-case demo segment specifically — it's now carrying more of the video's persuasive weight than any other single piece, so it's worth a few practice runs to make sure the reasoning trace actually displays cleanly on screen and the timing doesn't run long. If the live model's actual reasoning output doesn't read as cleanly as the scripted line above, it's reasonable to use the scripted version as an on-screen caption/overlay rather than relying on whatever text happens to stream out live — the point being demonstrated is the *capability*, and a clean caption is more watchable than a live stream of token-by-token output.

3.3 Public Project Link (mandatory — distinct field from the video)

A URL judges can use directly: either a live working demo (no login/paywall), or — more realistically for this project, given mock data sources — a public GitHub repo with setup instructions detailed enough to run from a clean clone. Since NightShift has no live public endpoint requirement, the repo link is the right choice here; just make sure `README.md` setup steps actually work (see §3.4).

3.4 Documentation — README.md (20 rubric points, NOT "Deployment Demonstration")

Correction from an earlier draft: this is the real 20-point category — Documentation, not deployment evidence. Per the official rubric, your GitHub submission's `README.md` must explain the problem, solution, architecture, setup instructions, and include relevant diagrams or images. Suggested structure:

1. **Safety guarantee headline** — lead with this, before the problem statement: **"Every draft remains in `staged` state until a human explicitly approves it. This is enforced at the database level."** Judges scan READMEs quickly; the headline hook (§0.5.1) should be impossible to miss in the first few lines, not discovered three sections in.
2. **Problem** — 2–3 sentences, same content as Writeup §3.1.2.
3. **Solution overview** — what NightShift does, in plain language.
4. **Data sources note** — state plainly, near the top: **"This version reads from a mock inbox. It's built so a real Gmail or Outlook connection could be swapped in later without changing the rest of the agent."** This is the single most likely question a judge has on first read, so it belongs early in the README, not buried in a setup appendix.
5. **Architecture diagram** — the TDD §2.2 graph, embedded as an image, not just described.
6. **Setup instructions** — exact commands from clean clone to running locally: dependencies, `.env` setup (with a placeholder, never a real key — see the API-key warning in §0), how to run the mock MCP servers, how to trigger a test overnight batch.
7. **Project structure** — a concrete directory tree, not just a description, so a judge skimming the repo orients in seconds:

```
nightshift/
├── agents/
│   ├── ingestion/        # IngestionAgent
│   ├── triage/            # TriageAgent
│   └── response/          # ResponseAgent
├── skills/                 # SKILL.md + skill logic per agent
├── mcp/                    # MCP server config + mock endpoints
├── memory/                 # tenant/property lookup store
├── features/               # Gherkin specs (Spec-Driven Development, TDD §2.5)
├── tests/                  # eval_urgency.py + fixtures, including the hard case
└── main.py                 # supervisor graph entrypoint
```

8. **(Optional) Deployment notes** — the Dockerfile/Cloud Run material from TDD §2.6, here rather than treated as its own scored section.

3.5 Legal & Submission Constraints (confirmed from Kaggle's official Competition Rules)

These are binding terms, not course concepts — worth a final pass before clicking submit:

- **Single submission:** each Team may submit only one (1) final Submission — there's no iterating after the fact, so the repo state at submission time is final.
- **Open-source obligation on winning:** if NightShift wins, the source code and submission must be licensed under CC-BY 4.0, which does not restrict commercial use. Don't include any dependency, dataset, or asset with a license that conflicts with this before submitting.
- **No official competition dataset is provided** — confirms the mock MCP endpoints (inbox, HOA portal, invoices) are the correct approach, not a placeholder to be replaced with "real" competition data later.
- **API keys/passwords must never appear in submitted code** — explicit, named in the rubric itself, not just general hygiene. Double-check the repo history too, not just the current file contents, since a key committed and later removed still exists in git history.
- **Deadline:** July 6, 2026, 11:59 PM PT. Draft/unsubmitted Writeups at the deadline are not considered by judges,

regardless of how complete the underlying code is.

End of PRD. See the companion Technical Design Document (TDD) for architecture, code structure, schemas, and implementation detail.